1 Norms

A norm measures vector size; must satisfy definiteness, homogeneity, triangle inequality.

$\|v\|_2 = \sqrt{\sum v_i^2}$, $\|v\|_1 = \sum |v_i|$, $\|v\|_\infty = \max |v_i|$

2 Errors & Condition

Absolute error $\Delta y = \tilde{y} - y$; relative error normalizes by scale:

$$\delta_y = \frac{\|\Delta y\|}{\|y\|}$$

Condition number = sensitivity of output to input perturbations (property of the *problem*, not the algorithm).

$$k_{\text{abs}} = |f'(x)|, \quad k_{\text{rel}} = \frac{|x|}{|y|}|f'(x)|$$

n-dim: $k_{\text{rel}}(x) = \max_j \left| \frac{\partial f}{\partial x_j} \frac{x_j}{f(x)} \right|$

Common cases:

$\sqrt{x}$: $k_{\text{rel}} = \frac{1}{2}$ $\quad \times$: $k_{\text{rel}} = 1$

$+$: $k_{\text{rel}} = \max\left\{ \left|\frac{x_1}{x_1+x_2}\right|, \left|\frac{x_2}{x_1+x_2}\right| \right\}$ $\rightarrow$ blows up under cancellation ($x_1 \approx -x_2$).

Linear operator L: stretch in worst direction times stretch of inverse:

$k_{\text{rel}}(L) = \|L\|_{X\to Y} \|L^{-1}\|_{Y\to X}$

Stability (opposed to condition) is a property of the *algorithm*: computed result should equal the exact solution of a slightly perturbed input.

3 Floating Point

Numbers are stored as $x = M \cdot B^E$ with base B, integer mantissa M, exponent E. Normalizing M to a fixed number of digits t gives (roughly) constant *relative* spacing.

Formal definition:

$$\mathbb{F}_{B,t} = \left\{ M \cdot B^E \;\middle|\; M, E \in \mathbb{Z},\ B^{t-1} \le |M| < B^t \ \text{ or } \ M = 0 \right\}$$

Bounding the exponent to make the set finite:

$$\mathbb{F}_{B,t,\alpha,\beta} = \{x \in \mathbb{F}_{B,t} \mid \alpha \le E \le \beta\}$$

(B = base, t = number of mantissa digits/precision, α, β = exponent range.)

$M_{\min} = B^{t-1}$, $M_{\max} = B^t - 1$

Relative spacing $\varrho = B^{1-t}$, $\quad$ machine precision $eps = \varrho/2$

Extremal representable numbers: $\sigma = B^{t-1}B^\alpha$ (smallest), $\lambda = (B^t - 1)B^\beta$ (largest)

Rounding: $\text{rd}(x) = x(1+\varepsilon)$, $|\varepsilon| \le eps$

4 Linear Systems

LR / Gauss

Gaussian elimination reduces A to upper-triangular R column by column; the multiplier used to zero out each entry below the pivot is stored as $l_{i,j}$. Collecting these multipliers into a unit-lower-triangular matrix L reproduces exactly the same elimination steps, giving the factorization $A = LR$. Once factored, $Ax = b$ reduces to two cheap triangular solves ($Ly = b$, then $Rx = y$) — useful when solving for many right-hand sides b.

$l_{i,j} = a_{i,j}^{(j)} / a_{j,j}^{(j)}$

$a_{i,k}^{(j+1)} = a_{i,k}^{(j)} - l_{i,j} a_{j,k}^{(j)}$

$A = LR$, $PA = LR$ (with row pivoting P if a pivot is zero/small)

Cost: $O(n^3/3)$, back-subst. $O(n^2)$

$$x_i = \frac{b_i - \sum_{j>i} a_{i,j} x_j}{a_{i,i}}$$

Condition of $Ax = b$

$\frac{\|\Delta x\|}{\|x\|} \leq k_{\text{rel}}(A) \frac{\|\Delta b\|}{\|b\|}$, $k_{\text{rel}}(A) = \|A\| \|A^{-1}\|$

Perturbed A: $\frac{\|\Delta x\|}{\|x\|} \leq \frac{k_{\text{rel}}(A)}{1 - k_{\text{rel}}(A) \frac{\|\Delta A\|}{\|A\|}} \left(\frac{\|\Delta A\|}{\|A\|} + \frac{\|\Delta b\|}{\|b\|} \right)$

Residual

Residual $\tilde{r} = b - A\tilde{x}$ measures how far a computed $\tilde{x}$ is from solving the system exactly; a small residual guarantees a small error only if A is well-conditioned.

$\tilde{r} = b - A\tilde{x} = -A\Delta x$

$\frac{1}{\|A^{-1}\|} \|\Delta x\| \leq \|\tilde{r}\| \leq \|A\| \|\Delta x\|$

QR / Givens

Instead of eliminating with row operations, multiply A from the left by orthogonal Givens rotation matrices $G_{j,i}$, each chosen to zero out one specific subdiagonal entry without disturbing already-zeroed entries. Orthogonal transformations never amplify errors ($\|Qx\| = \|x\|$), so this is more numerically stable than LR — at roughly twice the cost. The accumulated rotations give $Q^T A = R$ (upper triangular); solving $Ax = b$ becomes $y = Q^T b$, then $Rx = y$.

$Q^T Q = I$, $\det Q = \pm 1$

2×2 rotation: $G = \begin{bmatrix} c & s \\ -s & c \end{bmatrix}$

To zero $a_{i,j}$ using row j: $\rho = \pm\sqrt{a_{jj}^2 + a_{ij}^2}$, $c = \frac{a_{jj}}{\rho}$, $s = \frac{a_{ij}}{\rho}$

$Q^T A = R$, solve $y = Q^T b$, then $Rx = y$

Cost $\approx 2\times$ LR; works even if A is not square/regular

SVD

Decomposes the action of A into: rotate (V^T), scale each axis by σ_i (Σ), rotate again (U). Keeping only the largest few σ_i gives a cheap low-rank approximation of A.

$A = U\Sigma V^T = \sum_i \sigma_i u_i v_i^T, \;\; \sigma_1 \geq \sigma_2 \geq \cdots \geq 0$

Rank-r approx.: $\tilde{A} = \sum_{i=1}^{r} \sigma_i u_i v_i^T$

5 Least Squares

When A is $m \times n$ with $m > n$ (more equations than unknowns), $Ax = b$ generally has no exact solution; instead we minimize the residual norm.

$x^* = \arg\min \|Ax - b\|_2$

Setting $\nabla f = 0$ for $f(x) = \|Ax - b\|_2^2$ gives the **Normal Equation**:

$A^T A x^* = A^T b, \quad \nabla f = 2(A^T Ax - A^T b),\ H_f = 2A^T A \succ 0$

At the optimum the residual is orthogonal to the column space of A:

Residual $\perp$ image(A): $A^T \tilde{r}^* = 0$

Alternative: use $A = QR$, reduce to the triangular system $\|Rx - Q^T b\|_2$ — costs the same asymptotically but is numerically more stable than forming $A^T A$ directly.

Cost: Normal Eq. $O(mn^2{+}n^3)$; QR $O(mn^2{+}n^3)$ (QR more stable)

6 Interpolation

Given points $(x_i, f(x_i))$, find a function (usually a polynomial) that matches them exactly, to later evaluate/differentiate/integrate cheaply.

Lagrange

Build a basis polynomial $l_{j,n}$ that is 1 at node x_j and 0 at every other node; the interpolant is just a weighted sum of these.

$l_{j,n}(x) = \prod_{i \neq j} \dfrac{x - x_i}{x_j - x_i}$

$L(x) = \sum_j y_j l_{j,n}(x)$

Aitken–Neville

Builds the interpolation value at one point x recursively by blending lower-degree interpolants; efficient when only a single evaluation point is needed ($O(n^2)$, no explicit polynomial coefficients).

$P_{i,k}(x) = \dfrac{x - x_{i-k}}{x_i - x_{i-k}} P_{i,k-1}(x) + \dfrac{x_i - x}{x_i - x_{i-k}} P_{i-1,k-1}(x)$

Newton / Divided Diff.

Expresses the interpolant so that adding a new support point only adds one more term, without recomputing the rest — convenient when nodes are added incrementally.

$[x_{i-k}, \ldots, x_i]f = \dfrac{[x_{i-k+1}, \ldots, x_i]f - [x_{i-k}, \ldots, x_{i-1}]f}{x_i - x_{i-k}}$

$P_n(x) = \sum_{k=0}^{n} [x_0, \ldots, x_k] f \prod_{i<k} (x - x_i)$

Coincident nodes (Hermite/Taylor limit): $[\underbrace{x_0, \ldots, x_0}_{k+1}] f = \dfrac{f^{(k)}(x_0)}{k!}$

Error

$f(x) - P_n(x) = \dfrac{f^{(n+1)}(\xi)}{(n+1)!} \Phi_n(x)$, $\Phi_n(x) = \prod_i (x - x_i)$

Equidistant nodes: error bound grows badly (Runge's phenomenon):

$|\Phi_n(x)| \le n!\, h_n^{n+1} = \dfrac{n! H^{n+1}}{n^{n+1}} \approx O\big(\big(\tfrac{2}{e}\big)^n\big)$

Chebyshev nodes minimize the worst-case $|\Phi_n|$ instead, fixing this:

$x_i = \cos \dfrac{(2i+1)\pi}{2(n+1)}$, minimizes $\max |\Phi_n| = 2^{-n}$

Splines (cubic, order $k = 4$)

Instead of one high-degree polynomial (which oscillates), use low-degree (cubic) pieces glued together with matching value, slope, and curvature at each knot — avoids Runge's phenomenon entirely.

Degrees of freedom: $\dim \mathbb{P}_{k,\tau} = k + N - 1$

Local piece (Hermite blending form), $t = \frac{x - \tau_i}{h_i}$:

$s_i(x) = (1 - 3t^2 + 2t^3) y_i + (3t^2 - 2t^3) y_{i+1} + (t - 2t^2 + t^3) h_i y_i' + (t^3 - t^2) h_i y_{i+1}'$

C^2 continuity condition at each inner knot (unknowns: slopes y_i'):

$\frac{1}{h_{i-1}} y_{i-1}' + \left(\frac{2}{h_{i-1}} + \frac{2}{h_i} \right) y_i' + \frac{1}{h_i} y_{i+1}' = 3 \frac{y_{i+1} - y_i}{h_i^2} + 3 \frac{y_i - y_{i-1}}{h_{i-1}^2}$

Error: $\|f - s\|_\infty \le C h^4 \|f^{(4)}\|_\infty$

Missing 2 equations closed by boundary conditions: natural $s''(\tau_0) = s''(\tau_N) = 0$; clamped $y_0' = f'(a)$

FFT / DFT

Interpolates equally-spaced points on the unit circle with a trigonometric polynomial; the Fast Fourier Transform computes this (and its inverse) in $O(n \log n)$ instead of $O(n^2)$ by recursively splitting into even/odd indexed coefficients.

$\omega_n = e^{2\pi i / n}$, interpolation matrix $M = (\omega_n^{jk})$

$M^{-1} = \frac{1}{n} \overline{M}$, $\vec{c} = \frac{1}{n} \overline{M} \vec{v}$ (DFT: values $\to$ coefficients)

Divide-and-conquer (even/odd split), with $m = n/2$:

$v_j = \sum c_{2k} \omega_m^{kj} + \omega_n^j \sum c_{2k+1} \omega_m^{kj}$

$v_{m+j} = \sum c_{2k} \omega_m^{kj} - \omega_n^j \sum c_{2k+1} \omega_m^{kj}$

$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$

7 Quadrature

Approximate $I(f) = \int_a^b f(x)\, dx$ as a weighted sum of function evaluations $\tilde{I}(f) = (b - a) \sum_j c_j f(x_j)$; weights/nodes define the rule.

Basic rules ($[a, b]$)

Midpoint: $\tilde{I} = (b-a) f(\frac{a+b}{2})$

Trapezoid: $\tilde{I} = \frac{b-a}{2}(f(a)+f(b))$

Simpson: $\tilde{I} = \frac{b-a}{6}\big(f(a)+4f(\frac{a+b}{2})+f(b)\big)$

Errors (single interval)

Trapezoid: $I - \tilde{I} = -\frac{1}{12}(b-a)^3 f''(\zeta)$

Weights (general / Newton–Cotes)

Choose weights so every degree-$\leq m$ polynomial is integrated exactly (integrate the Lagrange basis polynomials):

$$c_j = \frac{1}{b-a}\int_a^b l_{j,m}(x)\,dx$$

Degree of accuracy $\geq m$ automatically; equidistant nodes $\Rightarrow$ weights can turn negative for $m \geq 8$ (unstable).

Gauss Quadrature

Instead of fixing nodes and solving for weights, also optimize the node *positions* — doubles the achievable exactness. Optimal nodes are the roots of the Legendre polynomial.

Max degree of accuracy $= 2m+1$ (nodes = zeros of $\tilde{P}_{m+1}$)

Legendre polynomials, orthogonal w.r.t. $\langle f, g\rangle = \int_{-1}^{1} f g\,dx$:

$$P_0 = 1,\ P_1 = x,\ P_n = \frac{2n-1}{n}xP_{n-1} - \frac{n-1}{n}P_{n-2}$$

Weights are always positive: $c_i = \frac{1}{2}\int_{-1}^{1} l_{i,m}^2\,dx > 0$

Composite

Split $[a, b]$ into N equal panels and apply a basic rule on each, summing the results — trades node count for accuracy.

$T(h) = h\Big[\frac{f(\tau_0)}{2}+\sum_{i=1}^{N-1} f(\tau_i)+\frac{f(\tau_N)}{2}\Big]$

Trap. error: $O(h^2) = O(N^{-2})$, deg.acc.$= 1$

Simpson error: $O(h^4) = O(N^{-4})$, deg.acc.$= 3$

Richardson / Romberg

Since the trapezoid error expands only in *even* powers of h (Euler–Maclaurin), combining $T(h)$ and $T(h/2)$ cancels the leading error term for free, giving Simpson's rule; repeating this recursively (Romberg) cancels ever more terms.

$S(h) = \frac{4}{3}T(h/2) - \frac{1}{3}T(h)$

Euler–Maclaurin: $T(h) - I(f) = c_1 h^2 + c_2 h^4 + \cdots + O(h^{2p+2})$

$$T_{i,0} = T(2^{-i}h),\ T_{i,k} = \frac{4^k T_{i,k-1} - T_{i-1,k-1}}{4^k - 1}$$

Error of $T_{i,k}$: $O(h^{2k+2})$

Monte Carlo

Approximate the integral (as an expectation) by averaging f at N random points — slow ($O(N^{-1/2})$) but the rate doesn't degrade with dimension, unlike grid-based rules.

$\bar{f} = \frac{1}{N}\sum f(x_i)$, $\mathbb{V}(\bar{f}) = \frac{1}{N}\mathbb{V}(f)$

Accuracy $O(N^{-1/2})$ (indep. of dimension d)

Tensor-product

Apply a 1D rule along each axis separately; accurate but cost grows exponentially in dimension.

Precision $O(m^{-2})$, cost $O(m^d)$

8 ODEs

An initial value problem specifies a curve via its derivative; we step forward in time approximating the unknown solution at discrete points $y^j \approx y(t_j)$.

IVP: $y'(t) = f(t, y(t))$, $y(t_0) = y^0$

Explicit Euler

Approximate $\int_{t_j}^{t_{j+1}} f\,ds$ by a rectangle using the left endpoint — cheap (one function evaluation), but only first order and can be numerically unstable for stiff problems.

$y^{j+1} = y^j + hf(t_j, y^j)$

Test eq. $y' = -\lambda y$: $y^{j+1} = (1 - h\lambda)^{j+1} y^0$

Implicit Euler

Same idea but evaluates f at the *new*, unknown point — requires solving an equation each step, but is far more stable.

$y^{j+1} = y^j + hf(t_{j+1}, y^{j+1})$

Test eq.: $y^{j+1} = \left(\dfrac{1}{1+h\lambda}\right)^{j+1} y^0$

Trapezoid (implicit, $p = 2$)

Averages the slope at both endpoints of the step; second order, but implicit (needs to solve for y^{j+1}).

$y^{j+1} = y^j + \frac{h}{2}\big(f(t_j, y^j) + f(t_{j+1}, y^{j+1})\big)$

Heun (explicit, $p = 2$)

Predictor-corrector: first guess y^{j+1} with explicit Euler, then plug that guess into the trapezoid formula — gets second order accuracy while staying fully explicit.

$\tilde{y}^{j+1} = y^j + hf(t_j, y^j)$

$y^{j+1} = y^j + \frac{h}{2}\big(f(t_j, y^j) + f(t_{j+1}, \tilde{y}^{j+1})\big)$

Runge–Kutta (general)

Generalizes Heun: evaluate f at several intermediate points s_l within the step, combine the resulting slopes k_l with tuned weights γ_l to cancel as many Taylor error terms as possible.

$s_l = t_j + \alpha_l h$, $k_l = f(s_l, u_l)$ (auxiliary directions)

$u_i = y^j + h \sum_l \beta_{i,l} k_l$ (stage values)

$y^{j+1} = y^j + h \sum_l \gamma_l k_l$ (update)

Consistency / Convergence

Consistency = error made in a *single* step (local); convergence = accumulated error over all steps (global). Consistency of order p implies convergence of order p.

$$\tau_{j,h} = \frac{y(t_{j+1}) - y^{j+1}}{h}$$

Consistent order $p \Rightarrow$ convergent order p

Explicit/Implicit Euler: $\tau_{j,h} = \frac{h}{2} y''(t_j) + O(h^2) \Rightarrow p = 1$